# Herald

# Protocol Research: MQTT 5.0

| Field | Value |
|---|---|
| Revision | 1 |
| Created | 2026-05-22 |
| Last modified | 2026-05-22 |
| Status | research-only (no implementation yet) |
| Status summary | MQTT 5.0 (OASIS standard, 2019) is the de-facto IoT pub/sub protocol — lightweight (2-byte header), TCP-based (or WebSocket), broker-mediated, QoS 0/1/2 delivery semantics. For Herald, MQTT is an INGRESS protocol: Herald subscribes to MQTT topics on an external broker (Mosquitto, EMQX, HiveMQ) and ingests messages as events for fan-out. NOT recommended for Herald to RUN a broker — too far from mission. Use Eclipse Paho Go client. P3 (opt-in). |
| Issues | open-questions: which broker is canonical for tests; QoS default; topic-to-tenant mapping. |
| Continuation | Wave 4f+ — open HRD-319..HRD-326 only if operator wants MQTT ingest in MVP. Else defer. |

## Constitutional anchors

- **§107 anti-bluff** — tests boot a real Mosquitto broker via `containers/` submodule; real `mosquitto_pub` publishes; Herald subscribes; assert receive.
- **§11.4.74 catalogue-check** — performed; no `vasic-digital` or `HelixDevelopment` MQTT module. Use `github.com/eclipse/paho.mqtt.golang` (Eclipse Foundation; established).
- **§11.4.61** — tracked doc.

## Table of contents

# §1. Protocol overview

**MQTT — Message Queuing Telemetry Transport.** OASIS standard. Current major version: **MQTT 5.0** (2019). MQTT 3.1.1 still widely deployed; 5.0 adds session expiry, request/response patterns, content-type, user properties.

**Wire format.** Binary. Fixed header is 2 bytes (control packet type + flags + remaining length). Variable header + payload per packet type.

**Transport.** TCP/IP (port 1883 plain; 8883 TLS). Also: MQTT over WebSocket (port 80/443 — useful for browser clients + cloud restrictions).

**Topology.** Broker-mediated pub/sub. Publishers send to topics; subscribers receive from matching topics. Wildcards (+ single-level, # multi-level).

**QoS levels.**

- **0** — at-most-once. Fire-and-forget. Fastest.
- **1** — at-least-once. PUBACK required. Possible duplicates.
- **2** — exactly-once. 4-way handshake. Slowest.

**Authentication.** Username/password (in CONNECT packet) + TLS client cert (mTLS). MQTT 5.0 adds enhanced auth via SASL.

**Retained messages.** Last message published to a topic is "retained" — new subscribers immediately receive it.

**Last Will and Testament (LWT).** Publisher declares a message at connect; broker publishes it if publisher disconnects abnormally.

**Adoption.** Pervasive in IoT (Tesla telemetry, Home Assistant, AWS IoT Core, Azure IoT Hub). Also widely used for sensors, smart-home, industrial monitoring.

# §2. Specification deep-dive

## §2.1 Packet types

- CONNECT / CONNACK
- PUBLISH / PUBACK / PUBREC / PUBREL / PUBCOMP (QoS dance)
- SUBSCRIBE / SUBACK
- UNSUBSCRIBE / UNSUBACK
- PINGREQ / PINGRESP
- DISCONNECT
- AUTH (MQTT 5.0)

## §2.2 Topic matching

```
/sensors/temperature/room1
/sensors/temperature/+    matches room1, room2, etc.
/sensors/#                matches everything under /sensors/
```

### §2.3 MQTT 5.0 features

- **Request/Response Topic** — publisher includes `Response Topic` property; subscriber publishes reply.
- **Content Type** — `application/cloudevents+json` declares CloudEvents payload.
- **User Properties** — arbitrary key/value (effectively HTTP-like headers).
- **Session Expiry** — broker holds session state for N seconds after disconnect.
- **Reason Codes** — granular error info (vs MQTT 3 single-byte CONNACK code).
- **Topic Aliases** — short integer aliases for long topics; saves bytes.
- **Shared Subscriptions** — `$share/group/topic` distributes messages across subscriber group.

## §3. Herald-specific applicability analysis

### §3.1 Use case

Herald INGESTS messages from MQTT brokers as events. Example:

- IoT fleet publishes telemetry to `org/{tenant_id}/devices/{device_id}/telemetry`.
- Herald subscribes to `org/+/devices/+/telemetry`.
- Each message becomes an event, dispatched via Herald's existing channels.

### §3.2 Herald is a CLIENT, not a broker

Herald should NOT run an MQTT broker. Brokers (Mosquitto, EMQX, HiveMQ, AWS IoT Core) are mature infrastructure; Herald subscribes as a client.

### §3.3 Multi-tenant topic mapping

Two approaches:

1. **Topic prefix carries tenant_id** — `org/{tenant_id}/...`. Herald subscribes with + wildcard and extracts tenant from topic.
2. **One MQTT connection per tenant** — each tenant configures broker + creds + topic root. Herald maintains N connections. Cleaner isolation; higher resource cost.

Recommend approach 2 for production (per-tenant credentials + audit).

### §3.4 JWT auth

MQTT doesn't speak JWT natively. Workarounds:

- Pass JWT as MQTT password — broker validates via custom auth plugin (EMQX supports this).
- Use mTLS client cert + map cert CN to tenant_id.
- Use MQTT 5.0 enhanced auth + SASL OAUTHBEARER.

Recommend: per-tenant MQTT credentials are PROVIDED BY THE OPERATOR (out-of-band); Herald uses them as-is.

### §3.5 Idempotency

MQTT QoS 1 can deliver duplicates. Herald MUST be idempotent — use the `PacketId` or a payload-embedded UUID as idempotency key (same Redis SETNX path).

### §3.6 CloudEvents integration

MQTT 5.0 supports CloudEvents payloads via Content Type property. Herald checks `Content Type: application/cloudevents+json`; parses + ingests. For MQTT 3.1.1 (no content type), Herald infers + heuristically parses.

### §3.7 Failure modes

1. **Broker disconnect.** Paho Go client auto-reconnects with exponential backoff.
2. **Subscription loss.** On reconnect, re-subscribe. CleanSession=false for durable subs.
3. **Backpressure.** If Herald can't keep up, broker queues (up to broker limit) then drops. Monitor lag metrics.
4. **Topic explosion.** Wildcard subscriptions can be expensive on the broker. Use shared subscriptions to spread load.

## §4. Step-by-step implementation guide for Herald

### §4.1 Add dep

`github.com/eclipse/paho.mqtt.golang` via `go get` or as submodule under `submodules/go-paho-mqtt/`. Recommend submodule per Helix §3.

### §4.2 New module `commons_mqtt/` OR new channel adapter

Two design options:

1. `commons_mqtt/` — generic MQTT ingest framework. Better if pherald + a future iherald-or-mherald both need MQTT.
2. `commons_messaging/channels/mqtt/` — channel adapter that implements the existing Channel interface, exposing MQTT as both an ingress (Subscribe()) and an egress (Send()).

Recommend option 2 — fits Herald's existing pattern.

### §4.3 Egress (publish)

Herald can also PUBLISH to MQTT topics as a fan-out channel — useful for delivering notifications to MQTT-subscribed devices/dashboards.

### §4.4 Subscriber config

Add to subscribers table:

- `mqtt_broker_url` — `tls://broker.example.com:8883`
- `mqtt_username` / `mqtt_password` (encrypted)
- `mqtt_topic_subscribe` — comma-separated list
- `mqtt_qos` — 0/1/2

### §4.5 New e2e_bluff_hunt invariants

- **E94** — boot Mosquitto via `containers/`; `mosquitto_pub -t 'org/abc/incidents' -m '{...}'`; Herald subscribed; assert event lands in events_processed.
- **E95** — QoS 1: simulate ack loss; assert duplicate handled idempotently.
- **E96** — CloudEvents payload: publish with `Content Type: application/cloudevents+json`; assert ce-id propagates through.
- **E97** — disconnect/reconnect: kill Mosquitto for 30s; restart; assert Herald reconnects + re-subscribes.

### §4.6 HRD scaffolding

- **HRD-319** — vendor paho.mqtt.golang.
- **HRD-320** — channel adapter `commons_messaging/channels/mqtt/`.
- **HRD-321** — connect manager + reconnect.
- **HRD-322** — subscription manager.
- **HRD-323** — CloudEvents payload handler.
- **HRD-324** — egress publisher.
- **HRD-325** — e2e_bluff_hunt E94–E97 + mutation gates + spec amendments.
- **HRD-326** — close Wave 4f (MQTT slice).

## §5. §107 anti-bluff testing strategy

**The bar:** real Mosquitto broker (booted via `containers/`); real `mosquitto_pub` CLI publishes; Herald subscribes + ingests; event lands in DB with correct fields.

### §5.1 Happy-path

- **Test 1: receive.** Mosquitto up; Herald subscribed to `test/topic`; `mosquitto_pub -t test/topic -m '{"k":"v"}'` → assert `events_processed` row exists within 5s with payload `{"k":"v"}`.
- **Test 2: publish.** Herald notify channel `mqtt` publishes to `test/out` → external subscriber receives.
- **Test 3: CloudEvents payload.** Publish with content-type CE → ce-id preserved.

### §5.2 QoS

- **Test 4: QoS 0.** Verify no PUBACK.
- **Test 5: QoS 1.** Verify PUBACK received.
- **Test 6: QoS 1 duplicate.** Send same msg twice with same PacketId → Herald idempotent.
- **Test 7: QoS 2.** Full 4-way handshake completes.

### §5.3 Failure modes

- **Test 8: broker down.** Kill Mosquitto; Herald's subscribe paused; restart; resub fires.
- **Test 9: bad credentials.** Wrong password → CONNACK error code logged; Herald backs off.
- **Test 10: cross-tenant.** Tenant A subscribes to Tenant B's topic prefix → no events delivered (broker ACL).

### §5.4 Performance

- **Test 11: throughput.** 10K msgs/sec sustained; lag < 1s.

### §5.5 Mutation gates

- Mutation: remove reconnect logic → Test 8 FAILS.
- Mutation: skip CE content-type detection → Test 3 FAILS.

### §5.6 Wire-level

- tcpdump :1883 → MQTT decoder; assert CONNECT + SUBACK + PUBLISH sequence.
- Use `mosquitto_sub -d` to verify Herald's published frames.

## §6. Open questions for operator

1. **Is MQTT ingest in scope for Herald MVP?** Recommend: NO. Operator-decides whether Wave 4f happens at all.
2. **Per-tenant connection or wildcard?** Recommend per-tenant for production.
3. **QoS default?** Recommend 1 (at-least-once + idempotency = exactly-once-effective).
4. **MQTT 5.0 only or 3.1.1 too?** Recommend 5.0 (newer brokers); 3.1.1 fallback for legacy.
5. **Egress also or only ingress?** Recommend both — egress is "free" once the client lib is in.

## §7. References

(All fetched 2026-05-22.)


- **MQTT 5.0 spec.** https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
- **MQTT 3.1.1 spec.** https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html
- **Eclipse Paho Go client.** https://github.com/eclipse-paho/paho.mqtt.golang
- **Pahod / paho.golang (MQTT 5 first-class).** https://github.com/eclipse-paho/paho.golang
- **Mosquitto broker.** https://github.com/eclipse/mosquitto
- **EMQX broker.** https://github.com/emqx/emqx
- **CloudEvents MQTT binding.** https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/bindings/mqtt-protocol-binding.md
- **HiveMQ MQTT 5 client encyclopedia.** https://www.hivemq.com/blog/mqtt-client-library-encyclopedia-golang/


## §8. Catalogue-check verdict

- **vasic-digital:** no module.
- **HelixDevelopment:** no module.

**Verdict: no-match → vendor** `github.com/eclipse/paho.mqtt.golang` **as submodule** under `submodules/go-paho-mqtt/`. Consider also `github.com/eclipse-paho/paho.golang` (newer MQTT-5-first library) — operator decides at implementation time.